

Керування секретами K8S

Огляд секретів

Щоб використовувати Secret, Pod має посилатися на Secret. Секрет можна використовувати з Pod трьома способами:

- Як файли в томі, змонтованому в одному або кількох його контейнерах.
- Як змінна середовища контейнера.
- За kubelet під час отримання зображень для Pod.

Ім'я секретного об'єкта має бути дійсним іменем субдомену DNS. Ви можете вказати `дані` та/або поле `stringData` під час створення файлу конфігурації для Secret. Поля `data` та `stringData` необов'язкові. Значення для всіх ключів у полі даних мають бути рядками в кодуванні base64. Якщо перетворення в рядок base64 небажане, ви можете вказати замість цього поле `stringData`, яке приймає довільні рядки як значення.

Ключі даних і `stringData` повинні складатися з буквено-цифрових символів, -, _ або .. Усі пари ключ-значення в полі `stringData` внутрішньо об'єднані в поле даних. Якщо ключ відображається як у полі даних, так і в полі `stringData`, значення, указане в полі `stringData`, має пріоритет.

Типи секретів

Під час створення Secret ви можете вказати його тип за допомогою поля типу ресурсу `Secret` або певних еквівалентних прапорців командного рядка `kubectl` (якщо доступно). Тип Secret використовується для полегшення програмної обробки секретних даних.

Kubernetes надає кілька вбудованих типів для деяких типових сценаріїв використання. Ці типи відрізняються залежно від виконуваних перевірок і обмежень, які Kubernetes накладає на них.

Ви можете визначити та використовувати власний тип Secret, призначивши непорожній рядок як значення типу для об'єкта Secret. Порожній рядок розглядається як непрозорий тип. Kubernetes не накладає жодних обмежень на назву типу. Однак, якщо ви використовуєте один із вбудованих типів, ви повинні відповідати всім вимогам, визначеним для

Builtin Type

Usage

`Opaque`

arbitrary user-defined data

`kubernetes.io/service-account-token`

service account token

`kubernetes.io/dockercfg`

serialized `~/.dockercfg` file

`kubernetes.io/dockerconfigjson`

serialized `~/.docker/config.json` file

`kubernetes.io/basic-auth`

credentials for basic authentication

`kubernetes.io/ssh-auth`

credentials for SSH authentication

`kubernetes.io/tls`

data for a TLS client or server

`bootstrap.kubernetes.io/token`

bootstrap token data

Непрозорі секрети

Непрозорий — тип секрету за замовчуванням, якщо він пропущений у файлі конфігурації `Secret`. Коли ви створюєте `Secret` за допомогою `kubectl`, ви використовуватимете загальну підкоманду, щоб вказати тип `Opaque Secret`. Наприклад, наступна команда створює порожній `Secret` типу `Opaque`

```
kubectl create secret generic empty-secret  
kubectl get secret empty-secret
```

Вихід виглядає так:

NAME	TYPE	DATA	AGE
empty-secret	Opaque	0	2m6s

Стовпець `DATA` показує кількість елементів даних, що зберігаються в `Secret`. У цьому випадку `0` означає, що ми щойно створили порожній секрет.

Секрети маркера облікового запису служби

Тип секрету `kubernetes.io/service-account-token` використовується для зберігання маркера, який ідентифікує обліковий запис служби. Використовуючи цей секретний тип, вам потрібно переконатися, що в анотації `kubernetes.io/service-account.name` встановлено ім'я наявного облікового запису служби. Контролер Kubernetes заповнює деякі інші поля, такі як анотація `kubernetes.io/service-account.uid` ключ маркера в полі даних, для якого встановлено фактичний вміст маркера.

У наведеному нижче прикладі конфігурації оголошено секретний маркер службового облікового запису:

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-sa-sample
  annotations:
    kubernetes.io/service-account.name: "sa-name"
type: kubernetes.io/service-account-token
data:
  # You can include additional key value pairs as you do with Opaque Secrets
  extra: YmFyCg==
```

Секрети маркера облікового запису служби

Під час створення `Pod` Kubernetes автоматично створює секрет облікового запису служби та автоматично змінює ваш `Pod` для використання цього `Secret`. Маркер облікового запису служби `Secret` містить облікові дані для доступу до API.

Автоматичне створення та використання облікових даних API можна вимкнути або замінити за бажанням. Однак якщо все, що вам потрібно зробити, це безпечний доступ до сервера API, це рекомендований робочий процес.

Перегляньте документацію `ServiceAccount`, щоб дізнатися більше про те, як працюють облікові записи служби. Ви також можете перевірити поля `automountServiceAccountToken` і `serviceAccountName` модуля, щоб отримати інформацію про посилання на обліковий запис служби з модулів.

Секрети конфігурації Docker

Ви можете використовувати одне з наведених нижче значень типу, щоб створити Secret для зберігання облікових даних для доступу до реєстру Docker для зображень.

- `kubernetes.io/dockercfg`
- `kubernetes.io/dockerconfigjson`

Тип `kubernetes.io/dockercfg` зарезервовано для зберігання серіалізованого `~/.dockercfg`, який є застарілим форматом для налаштування командного рядка Docker. Використовуючи цей секретний тип, ви повинні переконатися, що поле секретних даних містить ключ `.dockercfg`, значення якого є вмістом файлу `~/.dockercfg`, закодованого у форматі base64.

Тип `kubernetes.io/dockerconfigjson` призначений для зберігання серіалізованого JSON, який відповідає тим самим правилам форматування, що й файл `~/.docker/config.json`, який є новим форматом для `~/.dockercfg`. Під час використання цього типу Secret поле даних об'єкта Secret має містити ключ `.dockerconfigjson`, у якому вміст для файлу `~/.docker/config.json` надається як рядок у кодуванні base64.

Секрети конфігурації Docker

Нижче наведено приклад типу секрету `kubernetes.io/dockercfg` :

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-dockercfg
type: kubernetes.io/dockercfg
data:
  .dockercfg: |
    "<base64 encoded ~/.dockercfg file>"
```


Секрети конфігурації Docker

Коли ви створюєте ці типи секретів за допомогою маніфесту, сервер API перевіряє, чи існує очікуваний ключ у полі даних, і перевіряє, чи можна проаналізувати надане значення як дійсний JSON. Сервер API не перевіряє, чи справді JSON є конфігураційним файлом Docker.

Якщо у вас немає конфігураційного файлу Docker або ви хочете використовувати `kubectl` для створення секрету реєстру Docker, ви можете:

```
kubectl create secret docker-registry secret-tiger-docker \
  --docker-username=tiger \
  --docker-password=pass113 \
  --docker-email=tiger@acme.com
```

Секрети конфігурації Docker

Ця команда створює секрет типу `kubernetes.io/dockerconfigjson` . Якщо ви вивантажите вміст `.dockerconfigjson` із поля даних, ви отримаєте такий вміст JSON, який є дійсною конфігурацією Docker, створеною на льоту:

```
{
  "auths": {
    "https://index.docker.io/v1/": {
      "username": "tiger",
      "password": "pass113",
      "email": "tiger@acme.com",
      "auth": "dGlnZXI6cGFzcjExMw=="
    }
  }
}
```

Базовий секрет автентифікації

Тип `kubernetes.io/basic-auth` призначений для зберігання облікових даних, необхідних для базової автентифікації. При використанні цього типу Secret поле даних Secret має містити такі два ключі:

- `username`: ім'я користувача для автентифікації;
- `password`: пароль або маркер для автентифікації.

Обидва значення для двох наведених вище ключів є рядками в кодуванні base64. Ви, звичайно, можете надати чистий текстовий вміст за допомогою `stringData` для створення Secret.

Наступний YAML є прикладом конфігурації для базового секрету автентифікації:

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-basic-auth
type: kubernetes.io/basic-auth
stringData:
  username: admin
  password: t0p-Secret
```

Базовий секрет автентифікації

Базовий тип автентифікації Secret надається лише для зручності користувача. Ви можете створити Oracle для облікових даних, які використовуються для базової автентифікації. Однак використання вбудованого секретного типу допомагає уніфікувати формати ваших облікових даних, а сервер API перевіряє, чи надано необхідні ключі в секретній конфігурації.

Секрети автентифікації SSH

Вбудований тип `kubernetes.io/ssh-auth` надається для зберігання даних, які використовуються під час автентифікації SSH. Використовуючи цей секретний тип, вам доведеться вказати пару ключ-значення `ssh-privatekey` у полі даних (або `stringData`) як облікові дані SSH для використання.

Наступний YAML є прикладом конфігурації для секрету автентифікації SSH:

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-ssh-auth
type: kubernetes.io/ssh-auth
data:
  # the data is abbreviated in this example
  ssh-privatekey: |
    MIIEpQIBAAKCAQEaUlb/Y ...
```

Секрети автентифікації SSH

Секретний тип автентифікації SSH надається лише для зручності користувача. Ви можете створити Oracle для облікових даних, які використовуються для автентифікації SSH. Однак використання вбудованого секретного типу допомагає уніфікувати формати ваших облікових даних, а сервер API перевіряє, чи надано необхідні ключі в секретній конфігурації.

Застереження: особисті ключі SSH не встановлюють довіреного зв'язку між клієнтом SSH і хост-сервером самостійно. Для пом'якшення атак типу «людина посередині» потрібен додатковий засіб встановлення довіри, наприклад файл `known_hosts`, доданий до `ConfigMap`.

Секрети TLS

Kubernetes надає вбудований секретний тип `kubernetes.io/tls` для зберігання сертифіката та пов'язаного з ним ключа, які зазвичай використовуються для TLS. Ці дані в основному використовуються з завершенням TLS ресурсу Ingress, але можуть використовуватися з іншими ресурсами або безпосередньо робочим навантаженням. Під час використання цього типу секрету ключі `tls.key` і `tls.crt` мають бути надані в полі даних (або `stringData`) конфігурації секрету, хоча сервер API фактично не перевіряє значення для кожного ключа.

Наступний YAML містить приклад конфігурації для TLS Secret:

```
apiVersion: v1
kind: Secret
metadata:
  name: secret-tls
type: kubernetes.io/tls
data:
  # the data is abbreviated in this example
  tls.crt: |
    MIIC2DCCACGgAwIBAgIBATANBgkqh ...
  tls.key: |
    MIIEpgIBAAKCAQEA7yn3bRHQ5FHMq ...
```

Секрети TLS

Тип TLS Secret надано для зручності користувача. Ви можете створити `Opaque` для облікових даних, які використовуються для TLS-сервера та/або клієнта. Однак використання вбудованого типу Secret допомагає забезпечити узгодженість формату Secret у вашому проекті; сервер API перевіряє, чи надано необхідні ключі в секретній конфігурації.

Створюючи TLS Secret за допомогою `kubectl`, ви можете використовувати підкоманду `tls`, як показано в наступному прикладі:

```
kubectl create secret tls my-tls-secret \
  --cert=path/to/cert/file \
  --key=path/to/key/file
```

Пара відкритий/приватний ключ має існувати раніше. Сертифікат відкритого ключа для `--cert` має мати кодування .PEM (формат DER із кодуванням Base64) і відповідати вказаному закритому ключу для `--key`. Приватний ключ має бути у тому, що зазвичай називають форматом закритого ключа PEM, незашифрованим. В обох випадках початковий і останній рядки з PEM (наприклад, -----BEGIN CERTIFICATE----- і -----END CERTIFICATE----- для сертифіката) не включено.

Секрети початкового маркера

Маркер початкового завантаження Secret можна створити, явно вказавши тип Secret на `bootstrap.kubernetes.io/token`. Цей тип Secret призначений для маркерів, які використовуються під час процесу завантаження вузла. Він зберігає маркери, які використовуються для підпису добре відомих ConfigMaps.

Секрет маркера початкового завантаження зазвичай створюється в просторі імен системи kube та називається у формі `bootstrap-token-<ідентифікатор-токена>`, де `<ідентифікатор-токена>` — це рядок із 6 символів ідентифікатора маркера.

Як маніфест Kubernetes секретний маркер початкового завантаження може виглядати так:

```
apiVersion: v1
kind: Secret
metadata:
  name: bootstrap-token-5emitj
  namespace: kube-system
type: bootstrap.kubernetes.io/token
data:
  auth-extra-groups: c3lzdGVtOmJvb3RzdHJhcHB1cnM6a3ViZWFKbTpkZWZhdWx0LW5vZGutdG9rZW4=
  expiration: MjAyMC0wOS0xM1QwND0zOT0xMFo=
  token-id: NwVtaXRq
  token-secret: a3E0Z2lodnN6emduMXAwcg==
  usage-bootstrap-authentication: dHJ1ZQ==
  usage-bootstrap-signing: dHJ1ZQ==
```

Секрети початкового маркера

Завантажувальний тип Secret має такі ключі, указані під даними:

- `token-id`: Випадковий рядок із 6 символів як ідентифікатор маркера. Обов'язковий.
- `token-secret`: Випадковий рядок із 16 символів як фактичний секрет маркера. Обов'язковий.
- `description`: Зрозумілий для людини рядок, який описує, для чого використовується маркер.
Додатково.
- `expiration`: Абсолютний час UTC із використанням RFC3339 із зазначенням часу закінчення терміну дії маркера. Додатково.
- `usage-bootstrap-<usage>`: Логічний прапорець, що вказує на додаткове використання маркера початкового завантаження.
- `auth-extra-groups`: Розділений комами список імен груп, які будуть автентифіковані на додаток до групи `system:bootstrappers`.

Секрети початкового маркера

Наведений вище YAML може виглядати заплутаним, оскільки всі значення містяться в рядках у кодуванні base64. Фактично, ви можете створити ідентичний секрет, використовуючи такий YAML:

```
apiVersion: v1
kind: Secret
metadata:
  # Note how the Secret is named
  name: bootstrap-token-5emitj
  # A bootstrap token Secret usually resides in the kube-system namespace
  namespace: kube-system
type: bootstrap.kubernetes.io/token
stringData:
  auth-extra-groups: "system:bootstrappers:kubeadm:default-node-token"
  expiration: "2020-09-13T04:39:10Z"
  # This token ID is used in the name
  token-id: "5emitj"
  token-secret: "kq4gihvszzgn1p0r"
  # This token can be used for authentication
  usage-bootstrap-authentication: "true"
  # and it can be used for signing
  usage-bootstrap-signing: "true"
```

Створення секрету

Існує кілька варіантів створення секрету:

- створити секрет за допомогою команди `kubectl`
- створити секрет із конфігураційного файлу
- створити Secret за допомогою `kustomize`

Редагування секрету

Існуючий секрет можна редагувати за допомогою такої команди:

```
kubectl edit secrets mysecret
```

Це відкриє налаштований за замовчуванням редактор і дозволить оновити секретні значення в кодуванні base64 у полі даних:

```
# Please edit the object below. Lines beginning with a '#' will be ignored,  
# and an empty file will abort the edit. If an error occurs while saving this file will be  
# reopened with the relevant failures.  
#  
apiVersion: v1  
data:  
  username: YWRtaW4=  
  password: MmYyZDFlMmU2N2Rm  
kind: Secret  
metadata:  
  annotations:  
    kubectl.kubernetes.io/last-applied-configuration: { ... }  
  creationTimestamp: 2016-01-22T18:41:56Z  
  name: mysecret  
  namespace: default  
  resourceVersion: "164619"  
  uid: cfee02d6-c137-11e5-8d73-42010af00002  
type: Opaque
```

Використання секретів

Секрети можна монтувати як томи даних або виставляти як змінні середовища для використання контейнером у Pod. Секрети також можуть використовуватися іншими частинами системи без безпосереднього впливу на Pod. Наприклад, секрети можуть зберігати облікові дані, які інші частини системи повинні використовувати для взаємодії із зовнішніми системами від вашого імені.

Використання секретів як файлів із Pod

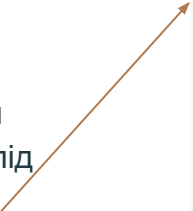
Щоб споживати секрет в обсязі в контейнері:

1. Створіть секрет або використовуйте наявний. Кілька модулів можуть посилатися на один секрет.
2. Змініть визначення Pod, щоб додати том у `.spec.volumes[]`. Назвіть том як завгодно, а поле `.spec.volumes[].secret.secretName` дорівнює назві об'єкта Secret.

Використання секретів

3. Додайте `.spec.containers[].volumeMounts[]` до кожного контейнера, якому потрібен секрет. Укажіть `.spec.containers[].volumeMounts[].readOnly = true` та `.spec.containers[].volumeMounts[].mountPath` до невикористаної назви каталогу, де ви хочете розмістити секрети.
4. Змініть зображення або командний рядок, щоб програма шукала файли в цьому каталозі. Кожен ключ у секретній карті даних стає іменем файлу під `mountPath`.

Це приклад Pod, який монтує Secret в тому:



```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
```

Використання секретів

Кожен секрет, який ви бажаєте використати, має бути вказаний у `.spec.volumes`.

Якщо в модулі є кілька контейнерів, то для кожного контейнера потрібен власний блок `volumeMounts`, але для кожного секрету потрібен лише один `.spec.volumes`.

Ви можете запакувати багато файлів в один секрет або використовувати багато секретів, залежно від того, що зручно.

Проекція секретних ключів на певні шляхи

Ви також можете керувати шляхами в тому томі, де проєктуються секретні ключі. Ви можете використовувати поле

`.spec.volumes[].secret.items`, щоб змінити цільовий шлях кожного ключа:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: mypod
      image: redis
      volumeMounts:
        - name: foo
          mountPath: "/etc/foo"
          readOnly: true
  volumes:
    - name: foo
      secret:
        secretName: mysecret
        items:
          - key: username
            path: my-group/my-username
```

Проекція секретних ключів на певні шляхи

Що станеться:

- секрет імені користувача зберігається у файлі `/etc/foo/my-group/my-username` замість `/etc/foo/username`.
- секретний пароль не проектується.

Якщо використовується `.spec.volumes[].secret.items`, проектуються лише ключі, указані в елементах. Щоб використати всі ключі з секрету, усі вони мають бути вказані в полі предметів. Усі перелічені ключі мають існувати у відповідному секреті. Інакше обсяг не створюється.

Проекція секретних ключів на певні шляхи

Дозволи секретних файлів

Ви можете встановити біти дозволу доступу до файлу для одного секретного ключа. Якщо ви не вкажете жодних дозволів, за замовчуванням використовується 0644. Ви також можете встановити режим за замовчуванням для всього секретного тому та замінити кожен ключ, якщо потрібно.

Наприклад, ви можете вказати режим за замовчуванням таким чином:

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: mypod
      image: redis
      volumeMounts:
        - name: foo
          mountPath: "/etc/foo"
  volumes:
    - name: foo
      secret:
        secretName: mysecret
        defaultMode: 0400
```

Проекція секретних ключів на певні шляхи

Потім секрет буде змонтовано в `/etc/foo`, а всі файли, створені під час монтування секретного тому, матимуть дозвіл `0400`.

Зауважте, що специфікація JSON не підтримує вісімкове позначення, тому використовуйте значення 256 для дозволів 0400. Якщо ви використовуєте YAML замість JSON для Pod, ви можете використовувати вісімкове позначення, щоб указати дозволи більш природним способом.

Зауважте, якщо ви виконуєте `kubectl` у Pod, вам потрібно перейти за символічним посиланням, щоб знайти очікуваний режим файлу. Наприклад,

Перевірте режим файлу секретів на модулі. `kubectl exec mypod -it sh`

```
cd /etc/foo
ls -l
```

Проекція секретних ключів на певні шляхи

Вихід подібний до цього:

```
total 0
lrwxrwxrwx 1 root root 15 May 18 00:18 password -> ../data/password
lrwxrwxrwx 1 root root 15 May 18 00:18 username -> ../data/username
```

Перейдіть за символічним посиланням, щоб знайти правильний режим фай

```
cd /etc/foo/..data
ls -l
```

Вихід подібний до цього:

```
total 8
-r----- 1 root root 12 May 18 00:18 password
-r----- 1 root root  5 May 18 00:18 username
```

Проекція секретних ключів на певні шляхи

Ви також можете використовувати зіставлення, як у попередньому прикладі, і вказати різні дозволи для різних файлів, як це:

У цьому випадку файл, результатом якого є `/etc/foo/my-group/my-username`, матиме значення дозволу `0777`. Якщо ви використовуєте JSON, через обмеження JSON ви повинні вказати режим у десятковій системі запису, `511`. Зауважте, що це значення дозволу може відображатися в десятковій системі запису, якщо ви прочитаєте його пізніше.

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
  volumes:
  - name: foo
    secret:
      secretName: mysecret
      items:
      - key: username
        path: my-group/my-username
        mode: 0777
```

Споживання секретних значень із томів

Усередині контейнера, який монтує секретний том, секретні ключі відображаються як файли, а секретні значення декодуються за допомогою base64 і зберігаються в цих файлах. Це результат команд, які виконуються всередині контейнера з прикладу вище:

```
ls /etc/foo/
```

Вихід аналогічний:

```
username  
password
```

```
cat /etc/foo/username
```

Вихід аналогічний:

```
admin
```

```
cat /etc/foo/password
```

Вихід аналогічний:

```
1f2d1e2e67df
```

Змонтовані секрети оновлюються автоматично

Коли секрет, який наразі використовується в томі, оновлюється, спроектовані ключі з часом також оновлюються. Під час кожної періодичної синхронізації kubelet перевіряє, чи є змонтований секрет свіжим. Однак kubelet використовує свій локальний кеш для отримання поточного значення Secret. Тип кешу можна налаштувати за допомогою поля `ConfigMapAndSecretChangeDetectionStrategy` у структурі `KubeletConfiguration`. Секрет можна поширювати за допомогою спостереження (за замовчуванням), на основі `ttl` або шляхом перенаправлення всіх запитів безпосередньо на сервер API. Як наслідок, загальна затримка від моменту оновлення секрету до моменту проектування нових ключів у Pod може дорівнювати періоду синхронізації kubelet + затримці поширення кешу, де затримка поширення кешу залежить від вибраного типу кешу (вона дорівнює затримці поширення, `ttl` кешу або нулю відповідно).

Використання секретів як змінних середовища

Щоб використовувати секрет у змінній середовища в модулі:

1. Створіть секрет або використовуйте наявний. Кілька модулів можуть посилатися на один секрет.
2. Змініть своє визначення Pod у кожному контейнері, який ви бажаєте споживати значення секретного ключа, щоб додати змінну середовища для кожного секретного ключа, який ви бажаєте споживати. Змінна середовища, яка використовує секретний ключ, має заповнити назву та ключ секрету в `env[].valueFrom.secretKeyRef`.
3. Змініть зображення та/або командний рядок так, щоб програма шукала значення у вказаних змінних середовища.

Це приклад Pod, який використовує секрети зі змінних середовища:

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
    - name: mycontainer
      image: redis
      env:
        - name: SECRET_USERNAME
          valueFrom:
            secretKeyRef:
              name: mysecret
              key: username
        - name: SECRET_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysecret
              key: password
      restartPolicy: Never
```

Споживання секретних значень зі змінних середовища

Усередині контейнера, який споживає секрет у змінних середовища, секретні ключі відображаються як звичайні змінні середовища, що містять декодовані base64 значення секретних даних. Це результат команд, які виконуються всередині контейнера з прикладу вище:

```
echo $SECRET_USERNAME
```

Вихід аналогічний:

```
admin
```

Вихід аналогічний:

```
echo $SECRET_PASSWORD
```

```
1f2d1e2e67df
```

Змінні середовища не оновлюються після секретного оновлення

Якщо контейнер уже використовує секрет у змінній середовища, контейнер не побачить оновлення секрету, доки його не буде перезапущено. Існують сторонні рішення для ініціювання перезапущів у разі зміни секретів.

Незмінні секрети

СТАН ФУНКЦІЙ: Kubernetes v1.19 [бета]

Бета-функція Kubernetes «Immutable Secrets and ConfigMaps» надає можливість установлювати окремі секрети та ConfigMaps як незмінні. Для кластерів, які широко використовують Secrets (принаймні десятки тисяч унікальних кріплень Secret to Pod), запобігання змінам їхніх даних має наступні переваги:

- захищає вас від випадкових (або небажаних) оновлень, які можуть спричинити збій програм
- покращує продуктивність вашого кластера, значно зменшуючи навантаження на kube-apiserver, закриваючи спостереження за секретами, позначеними як незмінні.

Незмінні секрети

Цією функцією керує шлюз функції `ImmutableEphemeralVolumes`, який увімкнено за замовчуванням, починаючи з версії 1.19. Ви можете створити незмінний секрет, встановивши для незмінного поля значення `true`. Наприклад,

```
apiVersion: v1
kind: Secret
metadata:
  ...
data:
  ...
immutable: true
```

Незмінні секрети

Використання `imagePullSecrets`

Поле `imagePullSecrets` — це список посилань на секрети в одному просторі імен. Ви можете використовувати `imagePullSecrets`, щоб передати секрет, який містить пароль реєстру зображень Docker (або інший) у `kubelet`. `Kubelet` використовує цю інформацію, щоб отримати приватне зображення від імені вашого Pod.

Організація автоматичного приєднання `imagePullSecrets`

Ви можете вручну створити `imagePullSecrets` і посилатися на нього з `ServiceAccount`. Будь-які модулі, створені за допомогою цього облікового запису `ServiceAccount` або створені за допомогою цього облікового запису `ServiceAccount` за замовчуванням, отримають у полі `imagePullSecrets` значення облікового запису служби.

Подробиці

Обмеження

Джерела секретних томів перевіряються, щоб переконатися, що вказане посилання на об'єкт дійсно вказує на об'єкт типу Secret. Тому секрет потрібно створити перед будь-якими модулями, які від нього залежать.

Секретні ресурси знаходяться в просторі імен. На секрети можуть посилатися лише модулі в тому самому просторі імен.

Розмір окремих секретів обмежений 1 MiB. Це робиться для того, щоб запобігти створенню дуже великих секретів, які виснажили б сервер API і пам'ять kubelet. Однак створення багатьох менших секретів також може виснажити пам'ять. Більш широкі обмеження на використання пам'яті через секрети є запланованою функцією.

Підобиці

Kubelet підтримує лише використання секретів для контейнерів, якщо секрети отримані з сервера API. Це включає будь-які модулі, створені за допомогою `kubectl` або опосередковано через контролер реплікації. Він не включає модулі, створені в результаті використання прапора `kubelet --manifest-url`, його прапора `--config` або REST API (це не звичайні способи створення модулів).

Секрети мають бути створені перед тим, як вони будуть використані в Pods як змінні середовища, якщо вони не позначені як необов'язкові. Посилання на секрети, яких не існує, перешкоджатимуть запуску Pod.

Посилання (поле `secretKeyRef`) на ключі, яких немає в іменованому Secret, не дозволять запустити Pod.

Підробиниці

Секрети, які використовуються для заповнення змінних середовища полем `envFrom`, ключі яких вважаються недійсними іменами змінних середовища, пропускатимуть ці ключі. Поду буде дозволено запуститися. Буде подія, причиною якої є `InvalidVariableNames`, і повідомлення міститиме список недійсних ключів, які були пропущені. У прикладі показано модуль, який посилається на типовий/мій секрет, який містить 2 недійсні ключі: `1badkey` і `2alsobad`.

```
kubectl get events
```

Вихід аналогічний:

LASTSEEN	FIRSTSEEN	COUNT	NAME	KIND	SUBOBJECT
0s	0s	1	dapi-test-pod	Pod	

TYPE	REASON	
Warning	InvalidEnvironmentVariableNames	kubelet, 127.0.0.1 Keys [1badkey, 2alsobad]

from the EnvFrom secret default/mysecret were skipped since they are considered invalid environment variable names.

Підробиниці

Взаємодія Secret і Pod протягом усього життя

Коли Pod створюється за допомогою виклику Kubernetes API, немає перевірки, чи існує секретний код, на який посилається. Щойно Pod заплановано, kubelet спробує отримати секретне значення. Якщо секрет неможливо отримати, оскільки він не існує або через тимчасову відсутність підключення до сервера API, kubelet періодично повторюватиме спроби. Він повідомить про подію про Pod із поясненням причини, чому він ще не запущений. Після отримання секрету kubelet створить і змонтує том, що містить його. Жоден з контейнерів Pod не запуститься, доки не буде змонтовано всі томи Pod.

Випадки використання

Випадок використання: як змінні середовища контейнера

Створіть секрет

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  USER_NAME: YWRtaW4=
  PASSWORD: MmYyZDFlMmU2N2Rm
```

Створіть секрет:

```
kubectl apply -f mysecret.yaml
```

Випадки використання

Використовуйте `envFrom`, щоб визначити всі дані Secret як змінні середовища контейнера. Ключ із Secret стає назвою змінної середовища в Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-test-pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      envFrom:
        - secretRef:
            name: mysecret
  restartPolicy: Never
```

Випадки використання

Випадок використання: модуль із ключами ssh

Створіть секрет, що містить кілька ключів ssh:

```
kubectl create secret generic ssh-key-secret --from-file=ssh-privatekey=/path/to/.ssh/id_rsa  
--from-file=ssh-publickey=/path/to/.ssh/id_rsa.pub
```

Вихід аналогічний: `secret "ssh-key-secret" created`

Ви також можете створити `kustomization.yaml` із полем `secretGenerator`, що містить ключі ssh.

Випадки використання

Тепер ви можете створити Pod, який посилається на секрет за допомогою ключа ssh і споживає його в тому:

Під час виконання команди контейнера частини ключа будуть доступні в:

```
/etc/secret-volume/ssh-publickey  
/etc/secret-volume/ssh-privatekey
```

Після цього контейнер може вільно використовувати секретні дані для встановлення з'єднання ssh.



```
apiVersion: v1  
kind: Pod  
metadata:  
  name: secret-test-pod  
  labels:  
    name: secret-test  
spec:  
  volumes:  
    - name: secret-volume  
      secret:  
        secretName: ssh-key-secret  
  containers:  
    - name: ssh-test-container  
      image: mySshImage  
      volumeMounts:  
        - name: secret-volume  
          readOnly: true  
          mountPath: "/etc/secret-volume"
```

Випадки використання

Випадок використання: модулі з обліковими даними для виробництва/тесту

Цей приклад ілюструє Pod, який використовує секрет, що містить облікові дані виробництва, і інший Pod, який споживає секрет з обліковими даними тестового середовища.

Ви можете створити `kustomization.yaml` із полем `secretGenerator` або запустити `kubectl create secret`.

```
kubectl create secret generic prod-db-secret --from-literal=username=produser  
--from-literal=password=Y4nys7f1l
```

```
secret "prod-db-secret" created
```

Вихід аналогічний:

Випадки використання

Випадок використання: модулі з обліковими даними для виробництва/тесту

Ви також можете створити секрет для облікових даних тестового середовища.

```
kubectl create secret generic test-db-secret --from-literal=username=testuser  
--from-literal=password=iluvtests
```

Вихід аналогічний: `secret "test-db-secret" created`

Випадки використання

Тепер зробить Pods:

```
cat <<EOF > pod.yaml
apiVersion: v1
kind: List
items:
- kind: Pod
  apiVersion: v1
  metadata:
    name: prod-db-client-pod
    labels:
      name: prod-db-client
  spec:
    volumes:
    - name: secret-volume
      secret:
        secretName: prod-db-secret
    containers:
    - name: db-client-container
      image: myClientImage
      volumeMounts:
      - name: secret-volume
        readOnly: true
        mountPath: "/etc/secret-volume"
- kind: Pod
  apiVersion: v1
```

```
  metadata:
    name: test-db-client-pod
    labels:
      name: test-db-client
  spec:
    volumes:
    - name: secret-volume
      secret:
        secretName: test-db-secret
    containers:
    - name: db-client-container
      image: myClientImage
      volumeMounts:
      - name: secret-volume
        readOnly: true
        mountPath: "/etc/secret-volume"
EOF
```

Випадки використання

Додайте поди до того самого kustomization.yaml:

```
cat <<EOF >> kustomization.yaml
resources:
- pod.yaml
EOF
```

Застосуйте всі ці об'єкти на сервері API, виконавши::

```
kubectl apply -k .
```

Обидва контейнери матимуть такі файли у своїх файлових системах зі значеннями для середовища кожного контейнера:

```
/etc/secret-volume/username
/etc/secret-volume/password
```

Зауважте, що специфікації для двох контейнерів відрізняються лише в одному полі; це полегшує створення модулів з різними можливостями на основі загального шаблону модуля.

Випадки використання

Ви можете ще більше спростити базову специфікацію Pod, використовуючи два облікові записи служби:

1. `prod-user` `3` `prod-db-secret`
2. `test-user` `3` `test-db-secret`

Специфікація Pod скорочена до:

```
apiVersion: v1
kind: Pod
metadata:
  name: prod-db-client-pod
  labels:
    name: prod-db-client
spec:
  serviceAccount: prod-db-client
  containers:
  - name: db-client-container
    image: myClientImage
```

Випадки використання

Випадок використання: dotfiles у секретному томі

Ви можете зробити свої дані "прихованими", визначивши ключ, який починається з крапки. Цей ключ представляє dotfile або "прихований" файл. Наприклад, коли такий секрет монтується в том, `secret-volume`:

Том міститиме один файл, який називається `.secret-file`, а контейнер `dotfile-test-container` буде мати цей файл у шляху `/etc/secret-volume/.secret-file`.

```
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
---
apiVersion: v1
kind: Pod
metadata:
  name: secret-dotfiles-pod
spec:
  volumes:
    - name: secret-volume
      secret:
        secretName: dotfile-secret
  containers:
    - name: dotfile-test-container
      image: k8s.gcr.io/busybox
      command:
        - ls
        - "-l"
        - "/etc/secret-volume"
      volumeMounts:
        - name: secret-volume
          readOnly: true
          mountPath: "/etc/secret-volume"
```

Випадки використання

Випадок використання: секрет, видимий для одного контейнера в модулі

Розглянемо програму, якій потрібно обробляти HTTP-запити, виконувати складну бізнес-логіку, а потім підписувати деякі повідомлення за допомогою HMAC. Оскільки він має складну логіку додатків, на сервері може виникнути непомічений експлойт віддаленого читання файлів, який може відкрити закритий ключ для зломисника.

Це можна розділити на два процеси в двох контейнерах: інтерфейсний контейнер, який обробляє взаємодію користувача та бізнес-логіку, але який не може бачити закритий ключ; і контейнер підписувача, який може бачити приватний ключ і відповідає на прості запити підписання з інтерфейсу (наприклад, через локальну мережу).

Завдяки цьому розділеному підходу зломиснику тепер потрібно обдурити сервер додатків, щоб він зробив щось довільне, що може бути важче, ніж змусити його прочитати файл.

Кращі практики

Клієнти, які використовують Secret API

Під час розгортання програм, які взаємодіють із секретним API, вам слід обмежити доступ за допомогою політик авторизації, таких як RBAC.

Секрети часто містять значення, які охоплюють спектр важливості, багато з яких можуть спричинити ескалацію в Kubernetes (наприклад, маркери облікових записів служби) і зовнішніх систем. Навіть якщо окрема програма може міркувати про силу секретів, з якими вона очікує взаємодії, інші програми в тому самому просторі імен можуть зробити ці припущення недійсними.

З цих причин запити на перегляд і перелік секретів у просторі імен є надзвичайно потужними можливостями, і їх слід уникати, оскільки перелік секретів дозволяє клієнтам перевіряти значення всіх секретів, які знаходяться в цьому просторі імен. Можливість перегляду та перегляду всіх секретів у кластері має бути зарезервована лише для найбільш привілейованих компонентів системного рівня.

Кращі практики

Програми, яким потрібен доступ до Secret API, повинні виконувати запити на отримання потрібних секретів. Це дозволяє адміністраторам обмежити доступ до всіх секретів, одночасно заносячи в білий список доступ до окремих примірників, необхідних додатку.

Для покращення продуктивності в порівнянні з циклічним отриманням клієнти можуть створювати ресурси, які посилаються на секрет, а потім переглядати ресурс, повторно запитуючи секрет, коли посилання змінюється. Крім того, також запропоновано API «масового перегляду», який дозволяє клієнтам переглядати окремі ресурси, і, ймовірно, він буде доступний у майбутніх випусках Kubernetes.

Властивості безпеки

Захист

Оскільки секрети можна створювати незалежно від модулів, які їх використовують, менший ризик розголошення секрету під час робочого процесу створення, перегляду та редагування контейнерів. Система також може вживати додаткових заходів безпеки щодо секретів, наприклад, уникати їх запису на диск, де це можливо.

Секрет надсилається до вузла лише в тому випадку, якщо цього вимагає Pod на цьому вузлі. Kubelet зберігає секрет у tmpfs, щоб секрет не записувався на дискове сховище. Після видалення модуля, який залежить від секрету, kubelet також видалить свою локальну копію секретних даних.

На одному вузлі можуть бути секрети для кількох модулів. Однак лише ті секрети, які запитує Pod, потенційно видимі в його контейнерах. Тому один Pod не має доступу до секретів іншого Pod.

Властивості безпеки

Захист

У контейнері може бути кілька контейнерів. Однак кожен контейнер у Pod має запитувати секретний том у своєму volumeMounts, щоб він був видимим у контейнері. Це можна використовувати для створення корисних розділів безпеки на рівні Pod.

У більшості дистрибутивів Kubernetes зв'язок між користувачами та сервером API, а також від сервера API до kubelets захищений протоколом SSL/TLS. Секрети захищені при передачі через ці канали.

СТАН ФУНКЦІЇ: Kubernetes v1.13 [бета]

Ви можете ввімкнути шифрування в стані спокою для секретних даних, щоб секрети не зберігалися у відкритому вигляді в etcd.

Властивості безпеки

Ризики

- На сервері API секретні дані зберігаються в etcd; тому:
 - Адміністратори мають увімкнути шифрування в стані спокою для даних кластера (потрібна версія 1.13 або новіша).
 - Адміністратори повинні обмежити доступ до etcd для користувачів із правами адміністратора.
 - Адміністратори можуть захотіти стерти/подрібнити диски, які використовує etcd, коли вони більше не використовуються.
 - Якщо etcd запущено в кластері, адміністратори повинні переконатися, що використовують SSL/TLS для однорангового зв'язку etcd.
- Якщо ви налаштуєте секрет за допомогою файлу маніфесту (JSON або YAML), який містить секретні дані, закодовані як base64, надання спільного доступу до цього файлу або його реєстрація у вихідному сховищі означає, що секрет зламано. Кодування Base64 не є методом шифрування та вважається таким же, як звичайний текст.
- Програми все ще повинні захистити значення секрету після прочитання його з тому, наприклад, щоб випадково не зареєструвати його або передати ненадійній стороні.
- Користувач, який може створити Pod, який використовує секрет, також може побачити значення цього секрету. Навіть якщо політика сервера API не дозволяє цьому користувачеві читати Secret, користувач може запустити Pod, який відкриває секрет.
- Наразі будь-хто з правами root на будь-якому вузлі може читати будь-який секрет із сервера API, імітуючи kubelet. Запланована функція надсилати секрети лише тим вузлам, яким вони дійсно потрібні, щоб обмежити вплив root-експлойту на один вузол.

Питання та відповіді